

Scaling: Distributed Training, FSDP, Deepspeed

AI Engineering From scratch #10

Data Parallelism

- Copy the entire model to N GPUs:
 - Split each training batch into N equal parts.
 - Each GPU runs a forward and backward on its shard of the data.
 - After the backward pass, average the gradients across all GPUs.
 - Every GPU updates its copy of the weights with the same averaged gradients, keeping all copies in sync.
- Pros: Linear throughput scaling. N GPUs process N times more data per step.
- Cons:
 - Every GPU holds a complete copy of the model, optimizer states, and gradients.
 - Data parallelism does nothing to reduce per-GPU memory. It only reduces training time.

Tensor Parallelism

- Split individual layers across GPUs.
- A single matrix multiplication is divided among GPUs, each computing part of the result.
 - Consider a weight matrix of shape (8192, 8192) in a feedforward layer. With 4-way tensor parallelism, each GPU holds a (8192, 2048) shard.
 - Each GPU multiplies the input by its shard, producing a partial result.
 - The partial results are combined (via all-reduce or all-gather) to produce the full output.
- Pros:
 - Reduces per-GPU memory for model weights. A 70B model split across 8 GPUs means each GPU holds ~8.75 parameters worth of weights.
- Cons: requires fast inter-GPU communication after every layer.

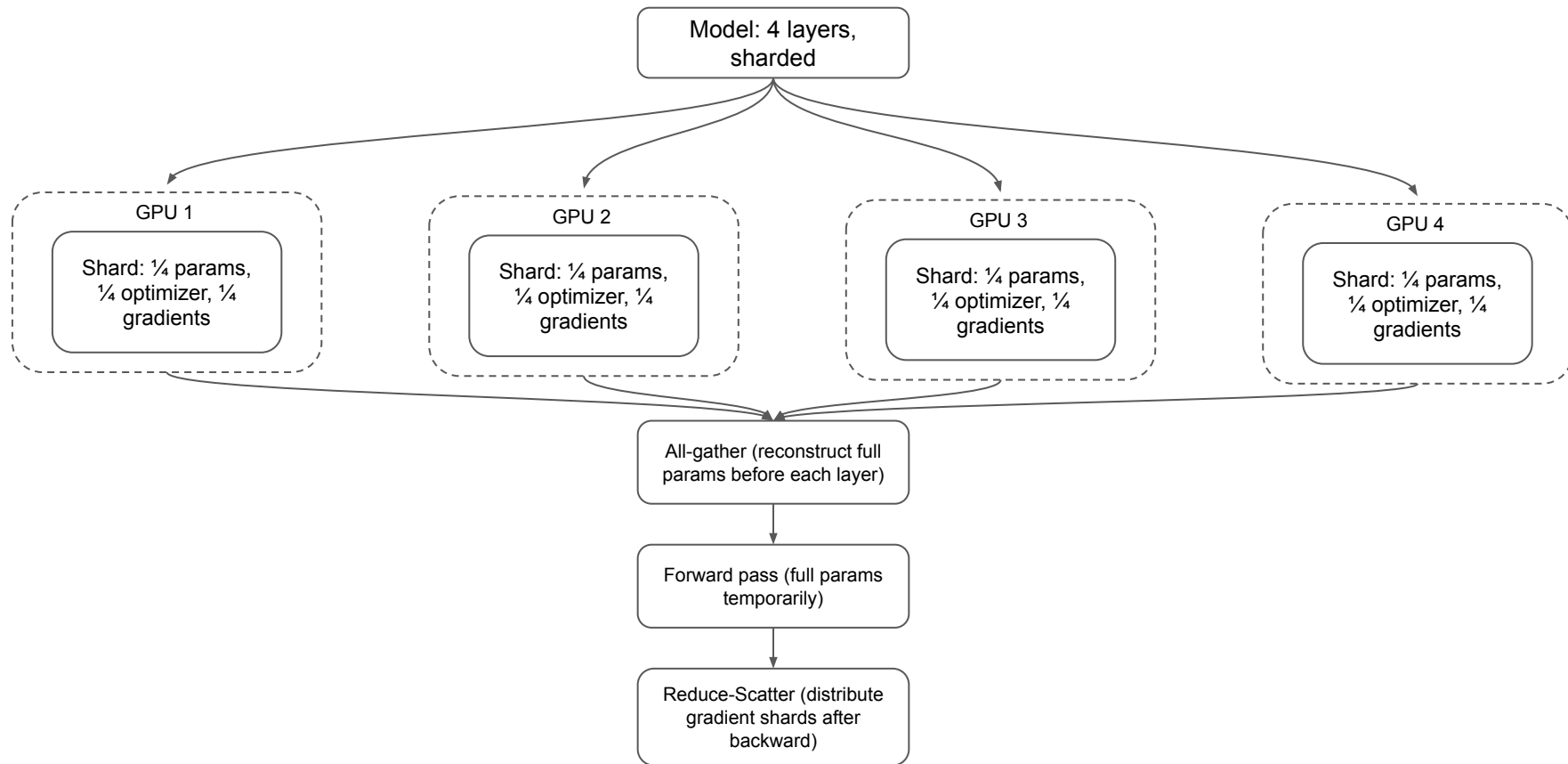
Pipeline Parallelism

- Split the model by layers.
 - GPU 1 runs layers 1-8. GPU 2 runs layers 9-16. GPU 3 runs layers 17-24, ...
 - Data flows through the pipeline: GPU 1 computes its layers and sends activation to GPU 2, which computes its layers and sends to GPU 3, and so on,
- Pros:
 - Minimal communication between GPUs – just the activations at layer boundaries, which are small compared to gradients or weights.
 - Works across nodes because bandwidth requirements are low.
- Cons:
 - Pipeline bubbles. When GPU 4 is computing the forward pass on micro-batch 1, GPUs 1, 2, and 3 are idle. During backward pass, the pattern reverses.
 - Solution: GPipe and PipeDream - splitting the batch into micro-batches. GPU 1 starts on micro-batch 2 as soon as it finishes forwarding micro-batch 1.

FSDP: Fully Sharded Data Parallel

- Combines the scalability of data parallelism with the memory efficiency of sharding.
 - Instead of each GPU holding a complete copy of the model, each GPU holds only $1/N$ of the parameters, gradients, and optimizer states.
 - Before a layer's forward pass, FSDP runs an all-gather to **collect the full parameters from all GPUs into each GPU's memory.**
 - After the forward pass, each GPU **discards the non-local parameters.**
 - During backward, the **all-gather runs again** to reconstruct parameters for gradient computation.
 - After the backward pass, a **reduce-scatter distributes gradient shards so each GPU only stores $1/N$ of the gradients.**

FSDP: Fully Sharded Data Parallel



DeepSpeed Zero

- is conceptually identical to FSDP but was developed independently by Microsoft.
- It defines 3 stages, each sharding more aggressively:
 - ZeRO-1: Shards optimizer stats only.
 - ZeRO-2: + Gradients.
 - ZeRO-3: + Parameters.
- ZeRO-3 is equivalent to FSDP.

Mixed Precision Training

- Modern training uses multiple floating-point formats simultaneously:
 - Forward pass: FP16 or BF16 (16-bit).
 - Master weights: FP32 (32-bit). Maintained by the optimizer for numerical precision during weight updates.
 - Loss scaling: Multiply the loss by a large constant before backward pass to prevent FP16 gradients from underflowing to zero. Divide by the same constant before the optimizer step.